VOLUME 4 OF 6

AI SOC, Observability, Red/Purple Teaming & Memory Security

Memory Governance, Agent Observability, AI Security Operations, Cognitive Security, Red Teaming, and Agent Reliability Engineering

Document Code: EASA-04
Version: 1.0
Date: June 2026
Scope: Domains 9, 10, 12, 18 & 19

Enterprise Agentic AI Security Architect (2026–2030) Master Research Program

Table of Contents

Executive Summary

Volumes 1 through 3 built the architecture, identity substrate, and governance scaffolding. This volume covers the part of the program that runs every day once the platform is live: watching what agents actually do, finding out where they will fail before an adversary or a customer does, and treating reliability as an engineering discipline rather than an aspiration.

A distinction worth making explicit up front, because the market's own terminology blurs it: AI observability platforms (LangSmith, Langfuse, Arize Phoenix, and similar tools covered in Domain 10) are built primarily for engineering quality — debugging why an agent looped, why a tool call failed, why output quality drifted. They are necessary infrastructure for an AI SOC but are not, by themselves, a security operations capability. An AI SOC additionally needs security-specific detection logic (is this trace a poisoning attempt, not just a quality regression), security-specific response playbooks (kill-switch invocation, credential revocation, blast-radius containment), and integration with the enterprise's existing SOC tooling and analysts. Most organizations buying observability tooling in 2026 are not yet buying AI SOC capability; this volume specifies the additional layer required to close that gap.

Domain 9 — Memory Security & Governance

Agent memory is both the platform's most valuable asset and one of its least governed. Unlike a conventional database, agent memory is written to continuously, by the agent itself, based on its own judgment about what is worth remembering — which means memory governance has to operate as a lifecycle discipline applied at write time, not as a periodic data-classification exercise applied after the fact.

9.1 Memory Types and Their Distinct Risk Profiles

Memory Type	What It Stores	Distinct Governance Concern
Episodic	Specific past interactions and events ("the user asked X on this date")	Highest privacy sensitivity — frequently contains PII tied to specific individuals and timestamps; primary target for right-to-erasure requests
Semantic	General facts and relationships learned across interactions, often vectorized	Aggregation risk — individually innocuous facts can combine into sensitive inferences; poisoning here has the broadest blast radius since it affects every future retrieval
Procedural	Learned patterns of how to perform tasks ("how this agent has historically handled this request type")	Behavioral drift risk — gradual procedural poisoning can shift agent behavior in ways that evade single-interaction anomaly detection
Long-Term	Persistent memory retained across sessions, often indefinitely by default	Retention-policy risk — the type most likely to violate data-minimization obligations if no expiry is enforced
Graph Memory	Entities and relationships represented as a connected graph	Inference risk — graph traversal can expose relationships never explicitly stated in any single memory entry, and poisoning a single well-connected node can corrupt many downstream inferences

9.2 Memory Threats

- **Poisoning —** malicious or false information is written into agent memory, either through direct manipulation or — more commonly in practice — through the agent itself ingesting and persisting poisoned content from an external source (a document, an email, a tool response), corresponding to ASI06 in the OWASP taxonomy (Volume 3).
- **Manipulation —** legitimate existing memory is altered to change agent behavior without necessarily introducing obviously false content — harder to detect than poisoning because the manipulated content may be plausible.
- **Leakage —** memory content is exposed to a party who should not have access, whether through a prompt-injection-driven exfiltration, an over-broad retrieval query, or simple misconfiguration of memory-store access controls.

- **Cross-Tenant Access —** in multi-tenant agent platforms, insufficient isolation allows one tenant's agent to read or influence another tenant's memory — the memory-layer instance of the MCP tenant-escape risk covered in Volume 2.
- **Context Corruption —** the agent's working context window, assembled from multiple memory and retrieval sources for a single task, becomes internally inconsistent or contradictory in ways that degrade reasoning quality even without a clear single point of poisoning.

9.3 Memory Governance Lifecycle

A defensible memory governance program enforces a consistent lifecycle for every memory write, rather than treating classification and retention as something applied later in a periodic batch process:

Stage	Action	Control
Create	Memory entry is written by the agent	Source attribution captured at write time — every memory entry is tagged with the originating session, tool call, or document
Classify	Sensitivity and data-type classification applied	Automated classification (PII detection, sensitivity scoring) applied synchronously at write time, not in a later batch job
Encrypt	Encryption applied per classification level	Field-level encryption for high-sensitivity entries; tenant-scoped encryption keys for multi-tenant stores
Retain	Retention period applied per classification and applicable regulation	Automated TTL (time-to-live) enforcement tied to data type and jurisdiction (GDPR, sector-specific retention rules)
Archive	Aged-out or superseded memory moved to lower-cost, access-restricted storage	Archive access requires elevated, logged authorization — not routinely queryable by the agent
Delete	Memory permanently removed at end of retention or on erasure request	Cryptographic erasure (key destruction) preferred over logical deletion for high-sensitivity data; deletion propagated to any derived embeddings or graph nodes
Audit	Every stage logged immutably	Full provenance and lineage trail retained independently of the memory content itself, so deletion of memory content does not also delete the audit trail of that memory's history

9.4 Provenance, Lineage, and the Right to Be Forgotten

Provenance (where did this memory entry originate) and lineage (what downstream memory, embeddings, or agent decisions were derived from it) are the two properties that make GDPR's right-to-erasure genuinely enforceable for agent memory rather than aspirational. Without lineage tracking, deleting a source memory entry does nothing to the semantic or graph-memory representations already derived from it — meaning the data subject's information persists in a different form, in violation of the erasure request's intent even if the letter of a narrow deletion has been satisfied. This is the single most common gap this program observes in

enterprise memory-governance implementations: deletion at the episodic-memory layer without corresponding deletion or invalidation at the semantic and graph-memory layers that were built from it.

Domain 10 — AI Observability & AI SOC

10.1 Observability Platform Landscape

The observability tooling market has matured quickly and consolidated around a recognizable set of patterns. None of these platforms is a security product first — they are quality and reliability engineering tools — but they are the substrate an AI SOC's security-specific detection logic is built on top of, because they are the systems already capturing the full trace data (every tool call, every model invocation, every retrieval) that security detection requires.

Platform	Primary Strength	Deployment Model	Best Fit
LangSmith	Deepest integration with LangChain/LangGraph; node-by-node state diffs and full execution-graph replay	Closed source; enterprise self-host available	Organizations standardized on LangGraph for orchestration
Langfuse	Leading open-source platform (MIT license); strong prompt management and evaluation tooling; large self-hosted community	Fully open source, self-hostable with no usage limits; acquired by ClickHouse (Jan 2026)	Organizations requiring full data sovereignty and self-hosting
Arize Phoenix	OpenTelemetry-native via OpenInference semantic conventions; ML-grade evaluation rigor	Open source (Elastic License 2.0)	Teams already on the broader Arize ML-observability platform; evaluation-heavy workflows
AgentOps	Agent-specific session replay and time-travel debugging; broad framework support	Python-only, cloud-only	Engineering teams needing lightweight, fast-to-deploy agent-specific monitoring
Helicone	Drop-in proxy-based logging with minimal integration overhead	Open source proxy	Quick request/response capture, especially for raw LLM call monitoring rather than full agent traces
OpenLLMetry / Traceloop	Vendor-neutral OpenTelemetry instrumentation standard for LLM and agent traces	Open source instrumentation layer, pairs with any OTel-compatible backend	Organizations wanting to avoid platform lock-in and feed traces into an existing OTel-based observability stack (Datadog, Honeycomb, New Relic)

Architectural recommendation

Standardize on OpenTelemetry / OpenLLMetry semantic conventions for trace instrumentation regardless of which front-end platform is chosen for day-to-day debugging. This keeps the raw trace data portable across the observability platform changes that are common in this fast-moving market, and — critically for this volume's purposes — lets the same trace stream feed both the engineering observability platform and the AI SOC's security detection layer without duplicating instrumentation.

10.2 AI SOC Architecture

An AI Security Operations Center extends the enterprise's existing SOC function with detection, triage, and response capability specific to agentic systems, rather than standing up an entirely parallel security organization. The architecture monitors seven surfaces, each requiring distinct detection logic because each fails differently:

Monitored Surface	What the AI SOC Watches For	Primary Signal Source
Agents	Goal drift, behavioral anomalies relative to declared purpose, autonomy-level violations	Trace data (Domain 10.1) correlated against the Agent Registry (Domain 14, Volume 3)
MCP	Tool poisoning indicators, unusual tool-call parameter patterns, unexpected new tool registrations	MCP gateway logs (Volume 2, Domain 5)
A2A	Unsigned or spoofed Agent Card usage, abnormal delegation patterns, unauthenticated endpoint access attempts	A2A gateway logs (Volume 2, Domain 6)
Memory	Anomalous write patterns suggesting poisoning, cross-tenant access attempts, classification policy violations	Memory governance audit log (Domain 9 of this volume)
Runtime	Sandbox escape attempts, unexpected egress, resource-consumption anomalies	Runtime/sandbox telemetry (Volume 1, Domain 7)
Guardrails	Guardrail bypass attempts, repeated near-miss triggers suggesting probing behavior	Guardrail platform logs (Domain 8 cross-reference, Volume 1)
Identity	Anomalous credential use, attestation failures, trust-score degradation events	SPIFFE/SPIRE and trust-broker logs (Volume 2, Domain 2 and 13)

The structural design choice that determines whether an AI SOC actually works is correlation across these seven surfaces, not monitoring each in isolation. A single anomalous tool call is noise; the same anomalous tool call following an unsigned Agent Card interaction and preceding an unusual memory write is a credible incident. This is precisely the cross-layer analysis MAESTRO calls for (Volume 1) applied operationally rather than at design time — the AI SOC is, in effect, MAESTRO's cross-layer threat model running continuously against live telemetry.

Domain 12 — AI Red Teaming

Red teaming an agentic system has to test for both conventional security flaws and agent-specific behavioral failure modes, and the two require different methodologies. A penetration test finds the SQL injection vulnerability in the tool an agent calls; it does not find the prompt-injection chain that gets the agent to call that tool with malicious parameters in the first place. Both are necessary.

12.1 Red Team Framework

A complete agentic red team exercise attacks across the same five surfaces the AI SOC monitors, using attack patterns specific to each:

- **Agents —** goal-hijack attempts via direct and indirect prompt injection (planted in documents, emails, web content the agent will retrieve), testing whether the Least Agency boundaries actually hold under adversarial pressure.
- **MCP —** tool poisoning injection into test tool descriptions, schema-manipulation payloads, attempts to trigger dynamic tool escalation beyond the originally granted scope.
- **A2A —** agent card spoofing and forgery attempts, agent-in-the-middle simulation, unauthenticated endpoint probing.
- **Memory —** poisoning injection through every available write path, cross-tenant access attempts, testing whether retention and erasure controls actually propagate through derived embeddings and graph memory as specified in Domain 9.
- **Runtime —** sandbox escape attempts, resource-exhaustion and budget-drain attacks, testing whether the kill switch (Domain 14, Volume 3) actually halts in-flight actions rather than only blocking new ones.
- **Identity —** credential theft and replay attempts, testing whether ephemeral credentials are genuinely ephemeral and whether compound-identity claims are validated rather than trusted at face value.

Promptfoo, DeepTeam, and similar open-source red-teaming frameworks have built test suites directly against the OWASP ASI categories (Volume 3, Domain 3.2), which is a sound starting point for automated, repeatable adversarial testing — but automated suites should be treated as a baseline coverage floor, not a substitute for human-led red team exercises that can chain multiple ASI categories together the way a real adversary would (for example, combining an ASI04 supply-chain foothold with an ASI07 inter-agent communication exploit to achieve an ASI08 cascading failure).

12.2 Purple Team Framework

Where red teaming finds gaps, purple teaming closes the loop by pairing every successful red-team finding with a corresponding detection pattern in the AI SOC and a tested response playbook — converting each exercise into a durable improvement rather than a point-in-time report.

Purple Team Output	Description	Owner
Detection Patterns	Specific telemetry signatures (trace patterns, log correlations) that would have caught the red-team's successful attack, implemented in the AI SOC's correlation logic	AI SOC / Detection Engineering

Purple Team Output	Description	Owner
Attack Simulations	Repeatable, automated versions of successful manual red-team findings, run on an ongoing cadence to catch regressions	Red Team, handed off to continuous testing pipeline
Response Playbooks	Documented, tested incident-response procedures for each successful attack pattern, including kill-switch invocation criteria and escalation paths	AI SOC / Incident Response, reviewed by AI CISO

Domain 18 — Cognitive Security

Cognitive security is the discipline specific to agentic AI that has no real precedent in traditional application security: protecting the integrity of the agent's reasoning process itself, not just the systems and data around it. This is the domain ASI01 (Goal Hijack) and ASI09 (Human-Agent Trust Exploitation) live in, and it is the domain where the gap between traditional security thinking and what agents actually require is widest.

- **Goal Hijacking —** an attacker manipulates the agent's interpreted objective, typically via prompt injection planted in content the agent will process as part of legitimate work (a planted instruction in a support ticket, a poisoned search result, a manipulated tool response).
- **Reasoning Manipulation —** an attacker influences the intermediate steps of the agent's reasoning (its chain-of-thought or planning trace) without necessarily changing its final stated objective, steering it toward a harmful path while appearing to remain on-task.
- **Planning Corruption —** for agents that generate explicit multi-step plans before execution, an attacker corrupts the plan-generation step itself, which is particularly dangerous because a corrupted plan can pass a superficial human review if the review only checks the stated goal rather than every step.
- **Cognitive Overload Attacks —** an attacker floods the agent's context with excessive, irrelevant, or contradictory information specifically to degrade reasoning quality and increase the probability of error or manipulation succeeding — the AI-native equivalent of a denial-of-service attack, but targeting reasoning quality rather than availability.
- **Agent Deception —** in multi-agent systems, one agent (compromised or adversarially designed) deliberately misrepresents information to another agent to manipulate the receiving agent's behavior — the mechanism underlying agent collusion identified in MAESTRO's ecosystem layer (Volume 1).

18.1 Agent Cognitive Integrity Framework

The control set for cognitive security differs structurally from conventional security controls because the attack surface is the model's own reasoning rather than a system boundary. The framework this program specifies has four components:

1. Instruction-data separation — architectural enforcement (not just prompting convention) that distinguishes trusted system instructions from untrusted retrieved content, so that content the agent processes cannot be interpreted with the same authority as its original instructions, regardless of how the content is phrased.
2. Plan validation before execution — for any agent generating an explicit plan, the full plan (not just the stated goal) is validated against policy before any step executes, catching planning corruption that a goal-only review would miss.
3. Reasoning trace capture — the agent's intermediate reasoning is captured and retained as part of the audit trail (Domain 4, Volume 1), making reasoning manipulation forensically reconstructable after an incident even when the final action appeared superficially reasonable.
4. Cross-agent skepticism — in multi-agent systems, information received from a peer agent (via A2A or shared memory) is treated with a lower default trust level than the agent's own direct observations, consistent with the trust-score discipline in Domain 13 (Volume 3), specifically to contain agent-deception attacks from propagating unchecked through a multi-agent pipeline.

Domain 19 — Agent Reliability Engineering (ARE)

Site Reliability Engineering gave software operations a quantitative vocabulary — SLIs, SLOs, error budgets — for treating reliability as an engineering discipline with explicit, negotiated tradeoffs against feature velocity. Agentic systems need the equivalent discipline, adapted for failure modes that classical SRE was never designed to measure: an agent can be perfectly available and perfectly fast while still being wrong, manipulated, or quietly drifting off its intended task.

19.1 Agent SLIs (Service Level Indicators)

SLI Category	Example Metric	Why It Matters Beyond Classical SRE
Goal conformance rate	% of completed tasks that matched the originally stated objective without unauthorized scope expansion	Directly measures Least Agency adherence and ASI01 exposure — a metric with no classical-SRE analog
Tool-call policy compliance	% of tool calls that passed policy evaluation on first attempt vs. required escalation or were blocked	Surfaces over-permissioning and policy drift before they become incidents
Memory integrity rate	% of memory writes passing provenance and classification checks without manual remediation	Leading indicator for poisoning risk (Domain 9), not just a quality metric
Human escalation accuracy	% of human-escalated decisions where escalation was actually warranted (precision) vs. missed escalations that should have occurred (recall)	Directly measures ASI09 exposure — both alert fatigue (over-escalation) and dangerous under-escalation are reliability failures
Cross-agent task success rate	% of delegated multi-agent tasks completing successfully without state poisoning or collusion-pattern detection	Captures the multi-agent failure modes (Volume 1, Domain 4.2) that single-agent SLIs miss entirely
Cost-per-successful-task	Total token/compute/tool cost divided by successfully completed tasks	Connects reliability directly to the FinOps domain (Volume 5); a runaway agent is simultaneously a reliability incident and a cost incident

19.2 Agent SLOs and Error Budgets

SLOs translate the SLIs above into negotiated targets ("goal conformance rate ≥ 99.5% measured over a rolling 30-day window") tied to the agent's assigned autonomy level (Domain 14, Volume 3): an L1 supervised-execution agent can tolerate a wider error budget because every action passes human review regardless; an L3 delegated-autonomy agent needs a substantially tighter error budget precisely because fewer human checkpoints exist to catch a failure before it has consequence. Error-budget burn should trigger an automatic autonomy-level downgrade — a defined, automated path back to a more supervised tier — rather than relying on a human noticing a dashboard trend, since the agents most likely to need this safeguard are, by definition, the ones operating with the least continuous human oversight.

19.3 Reliability Metrics and the ARE Framework

The complete ARE framework this program specifies sits at the intersection of three disciplines this volume has already covered: it consumes the observability trace data (Domain 10) to compute SLIs, it feeds error-budget status into the AI SOC's correlation logic (Domain 10.2) so reliability degradation and security incidents are visible in the same operational picture rather than two disconnected dashboards, and it uses purple-team-validated detection patterns (Domain 12.2) as a source of new SLIs whenever a red-team exercise surfaces a failure mode the existing metric set did not capture. Agent Reliability Engineering, done well, is not a separate function from AI security operations — it is the same operational discipline viewed through a reliability lens rather than a threat lens, and the two functions should share tooling, on-call rotations, and incident postmortems rather than operating as silos.